

FINAL YEAR PROJECT: TEAM VIVA & DEFENSE GUIDE

Campus Library Management System: Technical Q&A, Architecture & Demo Script

AUDIENCE: Project Team Members | PREPARATION: External Examiner & Guide Review

1. Team Member Role Allocations & Presentation Split

To deliver a confident, synchronized presentation in front of the examination committee, divide your defense responsibilities as follows:

PRESENTER 1: PROJECT LEAD & ARCHITECTURE

Core Topics: Problem Statement, Objectives, 3-Tier Architecture, Constraint Adherence (Zero Frameworks), Dual-Engine PostgreSQL design.

Live Demo Responsibility: Introductory remarks, System Topology diagram, and final conclusion.

PRESENTER 2: BACKEND, API & DATABASE

Core Topics: Node.js Express REST API, Relational Schema (3NF, cascades), JWT Authentication, RBAC (requireLibrarian guard), Security.

Live Demo Responsibility: Circulation desk actions, Overdue fine calculation logic, Database tables inspection.

PRESENTER 3: ADVANCED FEATURES (EXCEL & OPEN LIBRARY)

Core Topics: SheetJS binary Excel parsing & generation, bulk duplicate validation, Open Library (Internet Archive) live REST client.

Live Demo Responsibility: Live book search on openlibrary.org, 1-click import, downloading template & bulk student upload.

PRESENTER 4: MOBILE APP & INTEGRATION TESTING

Core Topics: Native Android Kotlin, Retrofit2 & OkHttp3 networking, ViewBinding, Material 3 UI, test suite results.

Live Demo Responsibility: Android Emulator walkthrough: Demo student login, catalog search, category chips, shelf location & loan countdowns.

2. 10-Minute Live Demonstration Walkthrough Script

Follow this exact sequential demonstration order to showcase all key capabilities without hiccups:

Step 1	Open Admin Portal	Navigate to <code>http://localhost:5000</code> in Chrome. Log in with <code>admin@library.com / admin123</code> . Show Overview metrics.
Step 2	Open Library Live Import	Go to "Book Catalog" -> click "Search Open Library". Search "Machine Learning". Show real covers. Click "Import".
Step 3	ISBN Auto-Fill Demonstration	Click "Add Book", enter ISBN "9780132350884", click "Auto-Fill". Watch Title, Author & Cover populate instantly.
Step 4	Excel Template Download	Go to "Students" tab -> click "Download Template". Open the downloaded .xlsx file to show binary formatting.
Step 5	Bulk Student Upload	Click "Upload Excel / CSV" and select a test roster. Show the modal feedback (e.g. "3 students created, 0 skipped").
Step 6	Android Mobile Client Demo	Open Android Emulator. Log in as Demo Student (<code>student@college.edu / student123</code>).
Step 7	Mobile Catalog & Search	Filter by category chips (Algorithms, Software Engineering), search "Clean Code", view shelf location Stack SE-1.
Step 8	Security & Student Restriction	Explain that the student app has NO "Issue Book" button. Show that loan issuance is strictly librarian-restricted.

FINAL YEAR PROJECT: TEAM VIVA & DEFENSE GUIDE

Section II: High-Frequency Professor Viva Questions & Ideal Answers

FOCUS: Examiner Defense, Edge Cases, Architectural Justifications

3. Anticipated Viva Questions & Technical Model Answers

Q: "Why did you choose Node.js and Vanilla JS instead of React or Angular?"

[Recommended Speaker: Presenter 1 (Lead)]

A: Our project strictly adheres to the fundamental technology constraints: Node.js, JavaScript, HTML, CSS, Kotlin, and PostgreSQL without third-party frameworks. Building the Admin Portal in Vanilla HTML5/CSS3/JS demonstrated deep mastery of DOM manipulation, asynchronous Fetch API, CSS Grid, and Single-Page Application (SPA) routing without framework bloat. It provides maximum execution speed and zero build dependencies.

Q: "How does your database handle PostgreSQL if the service is not installed on the professor's machine?" **[Recommended Speaker: Presenter 2 (Database)]**

A: We engineered a dual-engine database architecture in config/db.js. When external PostgreSQL on port 5432 is running, the system connects via a native connection pool (pg). If port 5432 is unavailable, our backend automatically activates an embedded persistent PostgreSQL engine (PGLite) that stores tables in backend/database/pgdata. This ensures zero-configuration portability for professors during grading while guaranteeing full PostgreSQL SQL syntax compliance.

Q: "How do you enforce that students cannot issue books to themselves?"

[Recommended Speaker: Presenter 2 (Backend)]

A: Security is enforced at the REST API middleware layer, not just the UI. Every circulation endpoint (POST /api/borrow/issue, POST /api/borrow/return/:id) is guarded by the requireLibrarian middleware. Even if a student crafts a direct HTTP request or uses Postman, the server verifies the JWT token payload. If req.user.role !== "librarian", the request is immediately aborted with HTTP 403 Forbidden.

Q: "How does the Open Library integration work, and what happens if the network is down?"

[Recommended Speaker: Presenter 3 (Advanced)]

A: In backend/services/openLibraryService.js, our backend acts as an authenticated proxy communicating with openlibrary.org/search.json and openlibrary.org/isbn/{isbn}.json. We parse the response, extract primary ISBN-13/10, resolve authors and high-res cover image IDs, and map them to our schema. If the external network fails, the service catches the error gracefully, returns meaningful error messages, and allows librarians to proceed with manual book additions.

Q: "How does bulk student Excel upload prevent duplicates and handle invalid data?"

[Recommended Speaker: Presenter 3 (Advanced)]

A: In backend/controllers/authController.js, we parse uploaded buffers using the SheetJS engine. We validate mandatory columns (name, email, student ID, password), normalize email and student ID casing, and check existing database records using SELECT queries. Duplicate emails or student IDs are automatically skipped and cataloged in a skipped report. Valid rows have passwords hashed with bcryptjs (10 rounds) before insertion.

Q: "How does the Android mobile client communicate with the backend, and how is session state managed?" **[Recommended Speaker: Presenter 4 (Mobile)]**

A: The Android app is built natively in Kotlin using Retrofit2 and OkHttp3 for RESTful communication, and Gson for JSON serialization. Upon successful login, the JWT token and user profile are saved to encrypted Android SharedPreferences via SessionManager. An AuthInterceptor automatically attaches the Authorization: Bearer <token> header to all subsequent requests. Material 3 ViewBinding renders the UI.

FINAL YEAR PROJECT: TEAM VIVA & DEFENSE GUIDE

Section III: Architecture Deep Dive, Troubleshooting & Checklist

FOCUS: Rapid Debugging, Project Handover, Confidence Checklist

4. Examiner Technical Deep Dive (Key File Reference)

When an examiner asks to "open the code and show me where X is implemented", immediately navigate to these exact files:

JWT & RBAC Middleware	backend/middleware/auth.js	Shows authenticateToken and requireLibrarian (403 guard).
Dual Database Engine	backend/config/db.js	Shows pg connection pool & PGlite persistent fallback logic.
Open Library Gateway	backend/services/openLibraryService.js	Shows fetch client calling openlibrary.org and cover URL builder.
Excel Bulk Processing	backend/controllers/authController.js	Shows uploadStudentsExcel and downloadStudentsTemplate with SheetJS.
Circulation & Fines	backend/controllers/borrowController.js	Shows issueBook, returnBook, overdue days calculation & \$0.50 fine math.
Android Networking	student-app/.../data/ApiService.kt	Shows Retrofit interface definitions and endpoints.
Android Session Auth	student-app/.../data/SessionManager.kt	Shows SharedPreferences JWT storage and user caching.
Admin SPA Interface	admin-web/js/app.js & index.html	Shows Open Library search modal, live table rendering, and upload dropzone.

5. Emergency Live Troubleshooting & Fallbacks

If any technical issue arises during the presentation, stay calm and execute these immediate 5-second remedies:

Issue: Backend Server Not Responding

Fix: Open terminal, run "cd backend && npm start". Server starts in under 2 seconds on port 5000.

Issue: Android App Cannot Connect to Server

Fix: On Android Emulator, 10.0.2.2 maps to localhost. Ensure Retrofit BASE_URL is "http://10.0.2.2:5000/api/".

Issue: Database Shows 0 Books After Reset

Fix: Run "node database/seedRealBooks.js" inside the backend folder to instantly restore the 8 Open Library books.

Issue: Examiner Asks for Fresh Database Reset

Fix: Run "npm run setup-db". Reinitializes schema, seeds Chief Librarian and Demo Student.

6. Final Team Presentation Day Checklist

- [X] Node.js server running in background on port 5000 (verified with /api/health).
- [X] Browser tab pre-opened to http://localhost:5000 logged in as Chief Librarian.
- [X] Android Studio Emulator running with Campus Library student app launched.
- [X] Sample Excel file (students_import_template.xlsx) downloaded on Desktop ready for live upload.
- [X] Two printed copies of "Professor_Project_Review_Report.pdf" placed on the review committee desk.